图形学物理仿真 Tutorial for Lab 1

概览

在 Lab 1 中你会**独立**实现一个刚体仿真框架。在这个框架中你会实现**显式 Euler** 的**长方体**刚体系统，你需要解决长方体之间的**碰撞检测**和**碰撞处理**。

你可以自由选择使用 C++ 开发环境（基于 VCX 框架）或 Python 开发环境（基于 Taichi 框架）。

你需要实现几个 demo 来展示你的模拟框架。

Demo 要求 (基础分16分)

1. 单个刚体
   - 实现一个长方体刚体的仿真，给定一个初始的速度/角速度，验证仿真的正确性。
   - 实现用户交互，通过鼠标或键盘给刚体施加外力。
2. 两个刚体碰撞
   - 初始化: 构造两个长方体碰撞的场景，可以直接给两个刚体相向的初速度保证他们相撞。相撞后他们可以正确的分离开。
   - 碰撞检测: taichi中我们提供了简单的长方体碰撞检测。vcx代码框架中需要通过调用第三方库 (fcl) 来实现长方体的碰撞检测
   - 碰撞处理: 基于上述碰撞检测, 实现课上讲过的 Impulse Based 碰撞处理方法。
   - 更多初始条件: 你可以设置不同的场景观察碰撞检测的结果。比如边-边碰撞和点-面碰撞分别如何返回碰撞法向和碰撞点的；如果有多个碰撞点 (面-面碰撞) 观测算法效果。
3. 复杂的场景
   - 构建一个至少有四个长方体的场景，他们之间会产生较多碰撞。
   - 实现简单的交互，比如鼠标拖动长方体，或为物体施加外力。
   - 你可以通过加入固定的地板和墙面，并加入重力 (前两个 Demo 不需要加重力) 来体现场景的复杂性。
   - 应当注意, 简单的冲量法（无持久接触处理）可能导致物体在地面上产生轻微“抖动”或无法完全静止。

Bonus 进阶挑战 （任选，3次lab的Bonus得分上限为12分）

B1. 多体牛顿摆 探索：你可以尝试模拟一排悬挂或放置的刚体。探究，简单的串行冲量处理能否实现能量的完美传递？如果发生“动能丢失”或传递顺序错误，可能的原因是什么？可尝试引入策略来提高准确性。

B2. 多体稳定堆叠 (Stacking)探索：尝试堆叠 3 层以上的长方体(考虑重力)。探究，物体能否稳定？是否会由于重力产生的微小震动而坍塌？可尝试引入策略来增强稳定性。

B3. 复杂几何支持：支持凸包 (Convex Hull) 或其他 Primitive（如球体、圆柱体）。

B4. 【困难】 约束动力学 (Constrained Dynamics) 探索：使用 Schur Complement 求解增广拉格朗日乘子，以硬约束方式同步解决所有碰撞与约束。

B5. 【困难】铰接刚体 (Articulated Bodies) 探索：通过点对点位置约束连接多个刚体，模拟机械臂或布娃娃系统。

B6. 【异常困难】学习并实现更健壮的 Continuous Collision Detection (CCD)，防止高速物体“穿墙”。

建议与提示

1. 刚体的朝向
   - 使用四元数来表示刚体的朝向，注意在更新刚体朝向时不保证更新后的四元数还是单位向量，记得在更新后要进行归一化。
   - 关于四元数具体的使用方法可以参考Eigen官方文档。
2. 转动惯量
   - 这里列举了常见物体的转动惯量，你也可以自行~~推导~~搜索。

vcx框架代码提示

1. 长方体的渲染
   - 对于单个长方体的渲染可以参考 Lab0 中的函数 `CaseBox::OnRender`。
   - 如果你选择用 `Eigen::Vector3f` 来表示刚体的位置，渲染时你可能需要如下函数帮助你进行类型转换：

```
static std::vector<glm::vec3> eigen2glm(Eigen::VectorXf const & eigen_v) {
    return std::vector<glm::vec3>(
        reinterpret_cast<glm::vec3 const *>(eigen_v.data()),
        reinterpret_cast<glm::vec3 const *>(eigen_v.data() + eigen_v.size())
    );
}
```

```
static Eigen::VectorXf glm2eigen(std::vector<glm::vec3> const & glm_v) {
    Eigen::VectorXf v = Eigen::Map<Eigen::VectorXf const, Eigen::Aligned>
(reinterpret_cast<float const *>(glm_v.data()), static_cast<int>(glm_v.size() * 3));
    return v;
}
```

2. 用户交互
   - Lab0 中的 `CaseBox::OnProcessMouseControl` 提供了简单的示例。
3. 碰撞检测
   - 我们不要求你实现一个碰撞检测算法，感兴趣的同学可以阅读参考资料了解算法和具体实现。
   - 你可以使用 The Flexible Collision Library (fcl) 实现你自己的碰撞检测，在 Lab0 中你应该已经安装了 fcl。你可以阅读 fcl 文档了解相关的类定义，或者阅读 fcl 提供的示例代码。我们提供一种可能的实现以供参考：

```
#include <fcl/narrowphase/collision.h>
void RigidBodySystem::collisionDetectBoxBox_fcl(int id1, int id2) {
    RigidBody const & b0 = _bodies[id1];
    RigidBody const & b1 = _bodies[id2];
    // Eigen::Vector3f RigidBody::dim - size of a box
    using CollisionGeometryPtr_t = std::shared_ptr<fcl::CollisionGeometry<float>>;
    CollisionGeometryPtr_t box_geometry_A(new fcl::Box<float>(b0.dim[0], b0.dim[1],
b0.dim[2]));
    CollisionGeometryPtr_t box_geometry_B(new fcl::Box<float>(b1.dim[0], b1.dim[1],
b1.dim[2]));
    // Eigen::Vector3f RigidBody::x - position of a box, Eigen::Quaternionf
RigidBody::q - rotation of a box
    fcl::CollisionObject<float> box_A(box_geometry_A,
fcl::Transform3f(Eigen::Translation3f(b0.x)*b0.q));
    fcl::CollisionObject<float> box_B(box_geometry_B,
fcl::Transform3f(Eigen::Translation3f(b1.x)*b1.q));
    // Compute collision - at most 8 contacts and return contact information.
    fcl::CollisionRequest<float> collisionRequest(8, true);
    fcl::CollisionResult<float> collisionResult;
    fcl::collide(&box_A, &box_B, collisionRequest, collisionResult);
    if(! collisionResult.isCollision()) return;
    std::vector<fcl::Contact<float>> contacts;
    collisionResult.getContacts(contacts);
    // You can decide whether define your own Contact
    for(auto const & contact : contacts) {
        _contacts.emplace_back(Contact(id1, id2, contact.pos, contact.normal,
contact.penetration_depth));
    }
}
```

taichi 代码提示

1. 在 Taichi 中没有内建四元数类型，建议使用 ti.Vector(4) 手动实现，并封装基本操作（乘法、归一化、更新）。
2. lab0代码中提供了简单的基于分离轴（Separating Axis Theorem， SAT）的长方体碰撞检测，对于基础分要求，使用SAT 即可。 如果需要更复杂的碰撞检测，在 Python 环境下，你可以安装 python-fcl。

   ```
   pip install python-fcl
   ```

   但请注意：FCL 是在 CPU 上运行的。如果你希望保持 Taichi 的并行优势，通常会在 Python 层面调用 FCL 获取碰撞点，然后写回 Taichi field。
3. 本地安装 taichi 后，命令行运行 ti gallery。里面有很多例子可以参考。虽无直接的刚体冲量法例子，但其 Data Layout 和 GGUI 渲染轮廓 的写法极具参考价值。

作业提交

作业须提交到**教学网**。

1. 代码管理：请确保你使用 Git 管理你的代码，在完成 Lab1 时输出你的提交记录：

```
git log --stat > lab1_log.txt
```

2. 清理记录：提交前执行 (VCX)

```
xmake clean -a
```

   或删除 **pycache** (Taichi)。 来移除所有二进制和临时文件。

3. 同时，也请提交一份作业报告，简要说明核心代码的思路，实现的交互方法，并展示你的 Demo。
4. 将你输出的提交记录、源代码和报告打包成 zip 压缩包，并以 `lab1_<your-student-ID>.zip` 命名，提交到教学网。

参考资料

- Eigen space transformations: Eigen 的三维变换文档，参考如何使用四元数
- fcl 官方文档
- Contact and friction simulation for computer graphics: Siggraph 2022课程，2.1.3讲了碰撞检测算法，后面几章介绍了几种处理碰撞和摩擦的方法